IoT Alarm Service – Architecture Document

---

1. Overview

This document describes the architecture of a **stateful Alarm Service** designed for a Linux-based IoT Gateway. The system processes real-time sensor data received via MQTT and evaluates configurable alarm rules to generate alerts.

The service supports:

- Threshold-based alarms
- Conditional alarms (with shunt conditions)
- Persistent state across reboots
- Scalable and modular processing

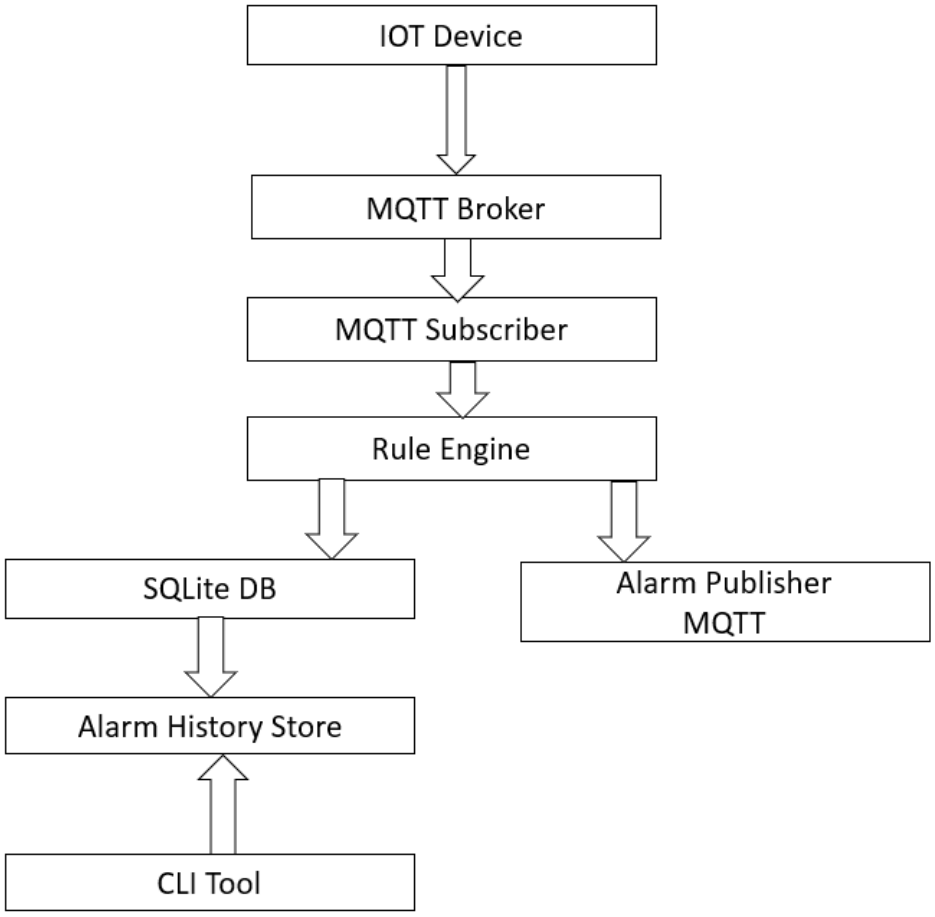
---

2. System Architecture

The system is composed of the following key components:

◈ Components

- IoT Devices (Sensors)
- MQTT Broker
- MQTT Subscriber (Data Ingestion Layer)
- Rule Engine
- State Manager (Database)
- Alarm Publisher
- CLI Interface


IOT Device
MQTT Broker
MQTT Subscriber
Rule Engine
SQLite DB
Alarm Publisher
MQTT
Alarm History Store
CLI Tool

3. Data Flow

1. IoT devices publish telemetry data (e.g., temperature, current) to MQTT topics.
2. The MQTT Subscriber listens to topics such as sensor/#.
3. Incoming data is parsed and stored temporarily in memory.
4. The Rule Engine evaluates the data against configured alarm rules.
5. The State Manager tracks ongoing alarm conditions and duration.
6. If conditions are met, an alarm is triggered.
7. The Alarm Publisher sends alarm notifications via MQTT.
8. Alarm events are stored in the database for history and monitoring.

---

4. Component Description

4.1 IoT Devices

Devices such as temperature sensors, energy meters, and CO_2 sensors publish data periodically using MQTT protocol.

4.2 MQTT Broker

The system uses Eclipse Mosquitto as the message broker.

Reason for selection:

- Lightweight and suitable for edge devices
- Low resource consumption
- Easy integration with Python

4.3 MQTT Subscriber (Data Ingestion)

- Subscribes to sensor/# topics
- Receives and parses JSON payloads
- Updates latest sensor values in memory

4.4 Rule Engine (Core Logic)

The Rule Engine evaluates incoming data based on predefined rules.

Supported Alarm Types:

1. Simple Threshold Alarm
Triggered when a value exceeds a threshold for a specified duration.
Example: Temperature > 24°C for 60 seconds.

2. Conditional Alarm
Triggered when:

- Primary condition is satisfied
- AND secondary (shunt) condition is satisfied

Example:
Temperature > 24°C AND Current > 0

4.5 State Manager (Persistence Layer)

The system uses SQLite for persistence.

Responsibilities:

- Store alarm rules
- Maintain active alarm states (start time, status)
- Store historical alarm events

Why SQLite:

- Serverless and lightweight
- Suitable for embedded systems
- Ensures persistence across system reboots

4.6 Alarm Processing Logic

- When a condition becomes true, the start time is recorded
- If the condition persists for the configured duration, an alarm is triggered
- If the condition fails before duration, the state is reset

4.7 Alarm Publisher

Once triggered, alarms are published to MQTT topics:

alarms/{alarm_id}

Example payload:

```
{
  "alarm_id": "temp_high",
  "status": "TRIGGERED",
  "timestamp": 1710000000
}
```

4.8 CLI Interface

A command-line interface is provided to:

- Add/update/delete alarm rules
- View active alarms
- View alarm history